

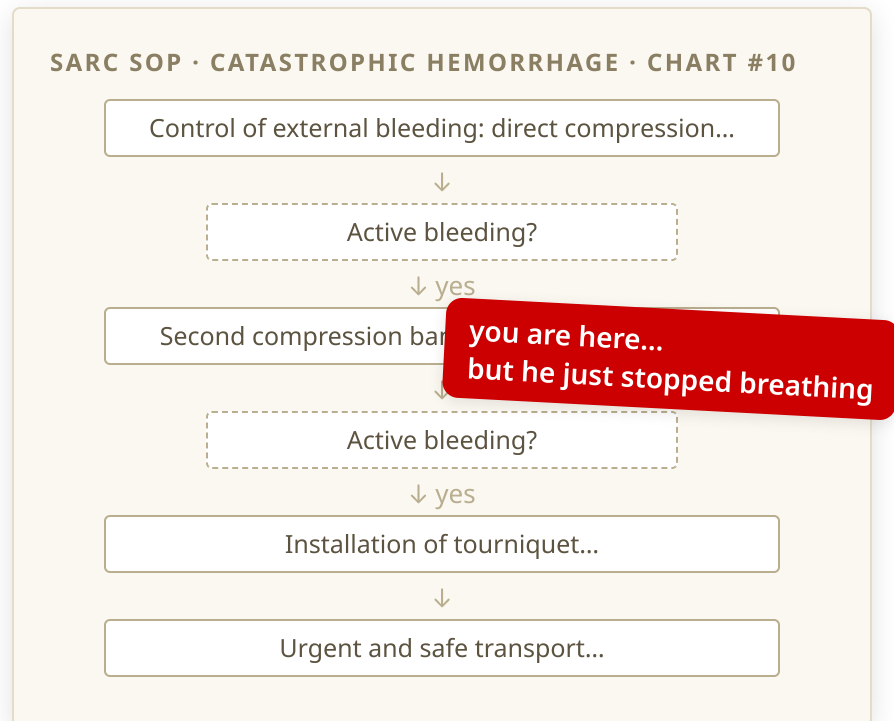
ResQRules

A knowledge-based system that walks a first responder through Red Crescent emergency protocols — and reacts when the patient doesn't follow the flowchart.

Knowledge-Based Systems · course project · built on the Experta forward-chaining engine

M.GH · M.SH · M.JA · M.Y.D · M.Marwan — supervised by Miss Omama

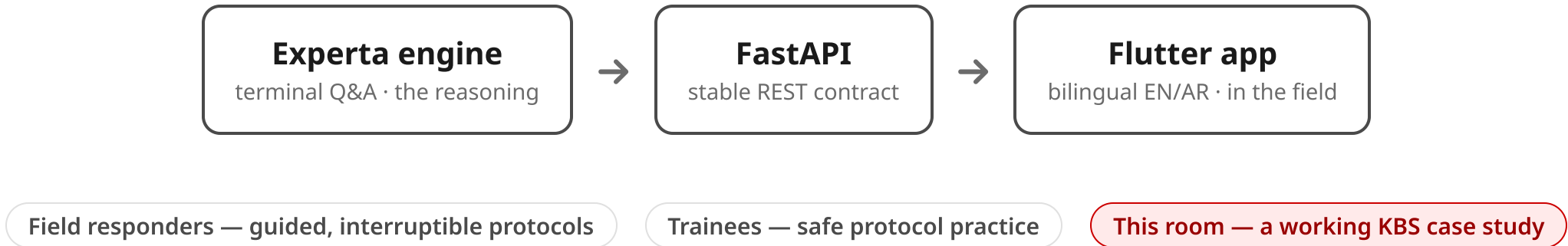
The patient doesn't follow the flowchart



- The Red Crescent SOP is **12 paper flowcharts** — expert, validated, and static.
- Under adrenaline, the responder must find the right chart, the right box, and follow the arrows correctly.
- Meanwhile the patient **deteriorates mid-protocol** — a new life threat appears while you're inside another box.

Paper can't re-route. That gap is what we built.

Protocols as software that reacts



Four assessment phases, as drawn on chart #1

1

Preliminary — SSS

Safety (staff · patient · bystanders)
Scene evaluation
Situation evaluation

2

Primary

AVPU consciousness
ABCDE: Airway · Breathing · Circulation · Disability · Expose

3

Secondary

History & vitals:
SAMPLER · OPQRST · GCS · PERRLA

4

Continuous

Reassess the patient; orient to the most appropriate care center

...and one exception, hard-coded as *structure*

1

Preliminary — SSS

Safety · Scene · Situation

2

Primary

AVPU + ABCDE

3

Secondary

SAMPLER · OPQRST · vitals

4

Continuous

Reassess & orient

Life-threatening bleeding? (gen_04)

Asked **between phases 1 and 2 — before ABCDE ever runs** — jumping straight to the Catastrophic Hemorrhage protocol. The chart encodes clinical precedence **positionally**.

Our engine encodes the same precedence as salience — coming up in Inference I.

Expert-validated knowledge — never our invention

SARC official SOP flowcharts

The 12-chart first-responder deck; 5 in scope for v1:

#1 General Assessment

#3 CPR Adult

#7 Heimlich / Choking

#8 Upper Airway

#10 Catastrophic Hemorrhage

An expert in the loop

M.GH — paramedic on the team. Collected the source charts, resolved ambiguities during extraction, and validated the transcribed knowledge. Knowledge acquisition with the expert at the table, not by email.

Rule zero: transcribe the chart — never "improve" medical content.

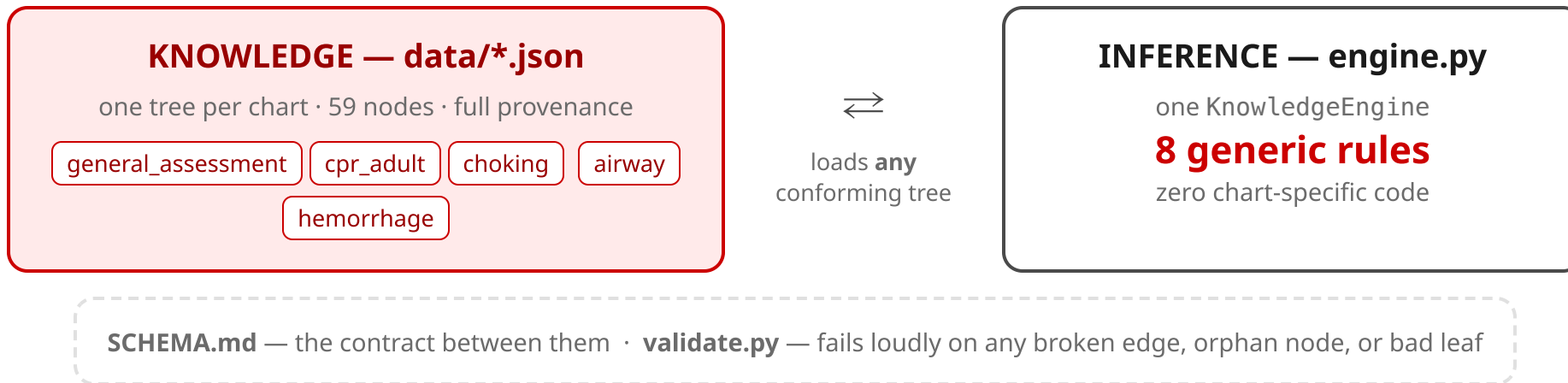
One @Rule per box is not a knowledge base



- Knowledge **trapped inside code** — every medical revision is a code change.
- The rule engine adds nothing: **no inference is demonstrated.**
- Experta reduced to a **flowchart costume.**

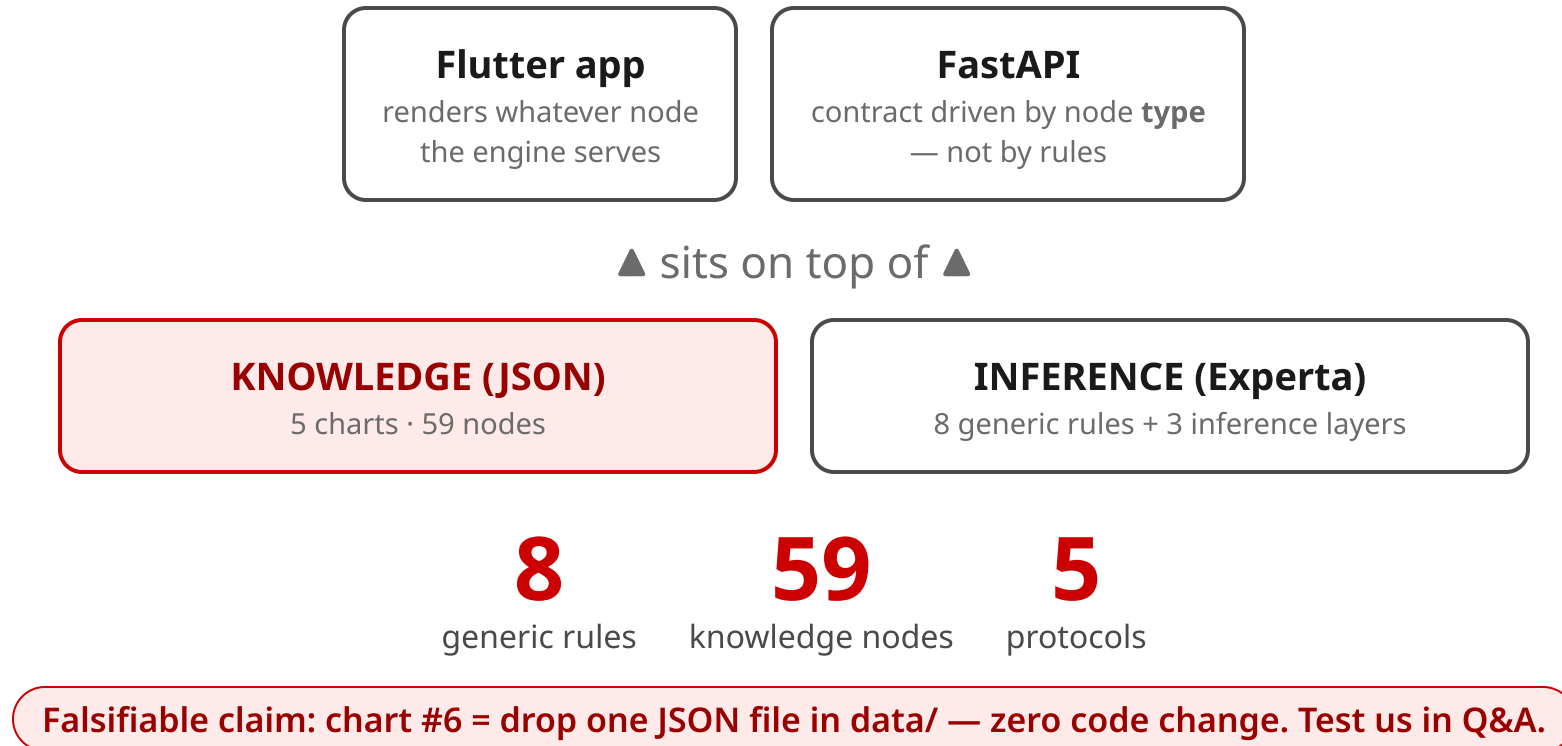
"Exactly like a web app with static hardcoded pages — and no database."

Knowledge is data; inference is generic

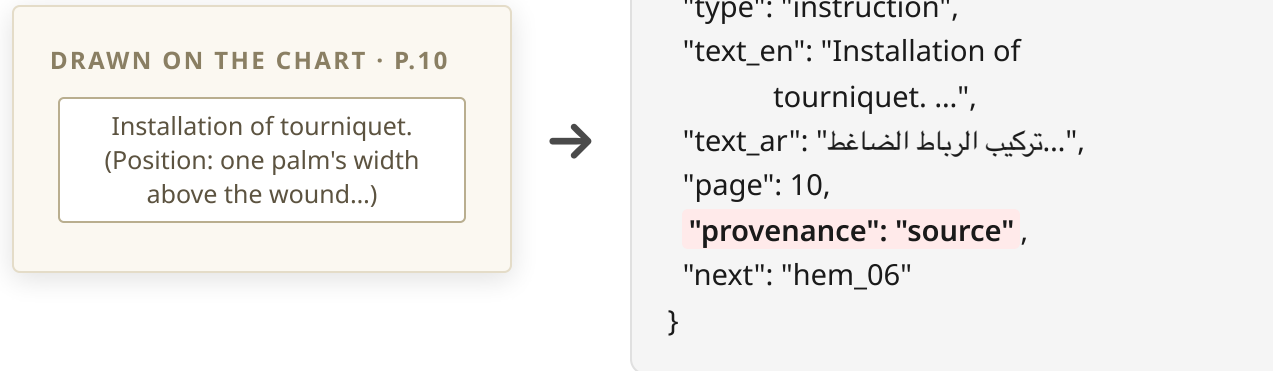


This separation is the textbook definition of a knowledge-based system.

...and the whole product stands on that separation



Faithfulness is engineered, not promised



- **Visual-only extraction** — pages rasterized @180 DPI and read as images. Text extraction drops the arrows, and **the arrows ARE the logic**.
- Every node cites its **source page**.
- Provenance audit trail: **58 of 59 nodes are source**; exactly 1 is added (navigation-only).
- `validate.py` **fails loudly**: broken edge, orphan node, bad leaf → non-zero exit.

Forward chaining = declare → match → fire

```
# the baseline walker — engine.py (real code, elided)
@Rule(AS.p << Position(nid=MATCH.nid),
      AS.a << Active(chart=MATCH.c),
      NOT(Bleeding()), salience=0)
def walk(self, p, a, nid, c):
    node = self.nodes[nid]    # ← read from the JSON
    ...                      # render node, ask the paramedic
    self._move(p, a, next_id) # retract old Position,
                             # declare the new one → RETE re-matches
```

Position

Active

Danger

Bleeding

StillBleeding

SignOfLife

— the six Fact types = the whole working memory

- One KnowledgeEngine subclass; charts are **facts about position**, not rules.
- Each answer **retracts / declares** a fact → the RETE network re-matches → the next rule fires. **That chain is the inference.**
- **8 rules total** walk all 59 nodes.

11 automated tests — every inference behavior is proven, not claimed

Salience encodes clinical precedence

100

ov_bleedingcatastrophic bleeding — key **b** · bleed-out kills fastest

90

ov_arrestnot breathing / no pulse — keys **n** · **p**

80

ov_unconsciousunconscious — key **u**

0

walk

the baseline chart walker

- At **every** prompt, the paramedic can assert a Danger fact **instead of answering**.
- Two dangers at once? **Conflict resolution**: the highest salience fires first — proven in TEST 1, not hand-sequenced.
- Guards: each override retracts its Danger (fires once) and never re-enters the protocol it's already in.

```
-- DANGER (type letter): [b]leeding [n]ot-breathing no-[p]ulse  
[u]nconscious  
!! DANGER [catastrophic_bleeding] (sal 100): OVERRIDE -> preempt,  
jump to 'hemorrhage' entry
```

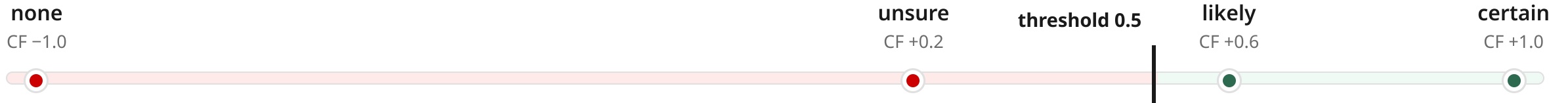
verbatim engine output · presentation/traces/override_bleeding_en.txt

A paper chart cannot accept an asynchronous fact.

Uncertainty is data too

cpr_11 — "[1] Sign of life?"

"Gasping is not considered efficient breathing" — chart #3, p.3



← below 0.5 → CPR branch (cpr_14) — "if in doubt, compress"

≥ 0.5 → ventilation branch (cpr_12) →

- Experta has **no native CF** → we model it as data: a `SignOfLife(cf)` fact + **one thresholding rule** (salience 20).
- The conservative bias lives **in the scale**: genuine doubt scores 0.2, far below the cut. Endpoints match a plain yes/no — only the ambiguous middle re-routes.

```
?? Confidence the patient has a SIGN OF LIFE
(breathing/pulse)?
3) unsure (CF +0.20)
~ CF +0.20 < +0.50: doubt -> CPR branch (if in doubt,
compress)
```

verbatim engine output · presentation/traces/cf_unsure_en.txt

GENERAL ASSESSMENT — CHART #1



WORKING MEMORY

Active(chart=hemorrhage)

RULE FIRED

Δ THIS STEP

- ~~Active(chart=general_assessment~~

```
+ Active(chart=hemorrhage) declare
```

ABCDE never ran — circulation-first, as structure.

ENGINE OUTPUT (VERBATIM)

```
[gen_05] Catastrophic Hemorrhage.  
>> JUMP to chart 'hemorrhage'  
(node 'hem_01') <<
```

real session ·
presentation/traces/walkthrough_yes_branch_en.txt


```
graph LR; A["Life-threatening bleeding? gen_04"] --> B["✓ no"]; B --> C["Primary Assessment — AVPU + ABCDE gen_06"]; C --> D["Stable or unstable? gen_07"]; D --> E["Unstable: immediate action, BLS gen_09"]; E --> F["Secondary Assessment — SAMPLER · OPQRST gen_10"]; F --> G(["Continuous Assessment & Orientation gen_11"]);
```

The flowchart illustrates the SAMPLER algorithm for trauma patients. It begins with a decision point: "Life-threatening bleeding?" (gen_04). If the answer is "no" (indicated by a green checkmark), the process moves to "Primary Assessment — AVPU + ABCDE" (gen_06). From there, it asks "Stable or unstable?" (gen_07). If the patient is unstable, it leads to "Unstable: immediate action, BLS" (gen_09). If stable, it proceeds to "Secondary Assessment — SAMPLER · OPQRST" (gen_10). Finally, the process concludes with "Continuous Assessment & Orientation" (gen_11), which is highlighted in a green oval.

No new fact types here — this is the plain walk at salience 0.

► ...and the YES branch is where the inference lives — watch it

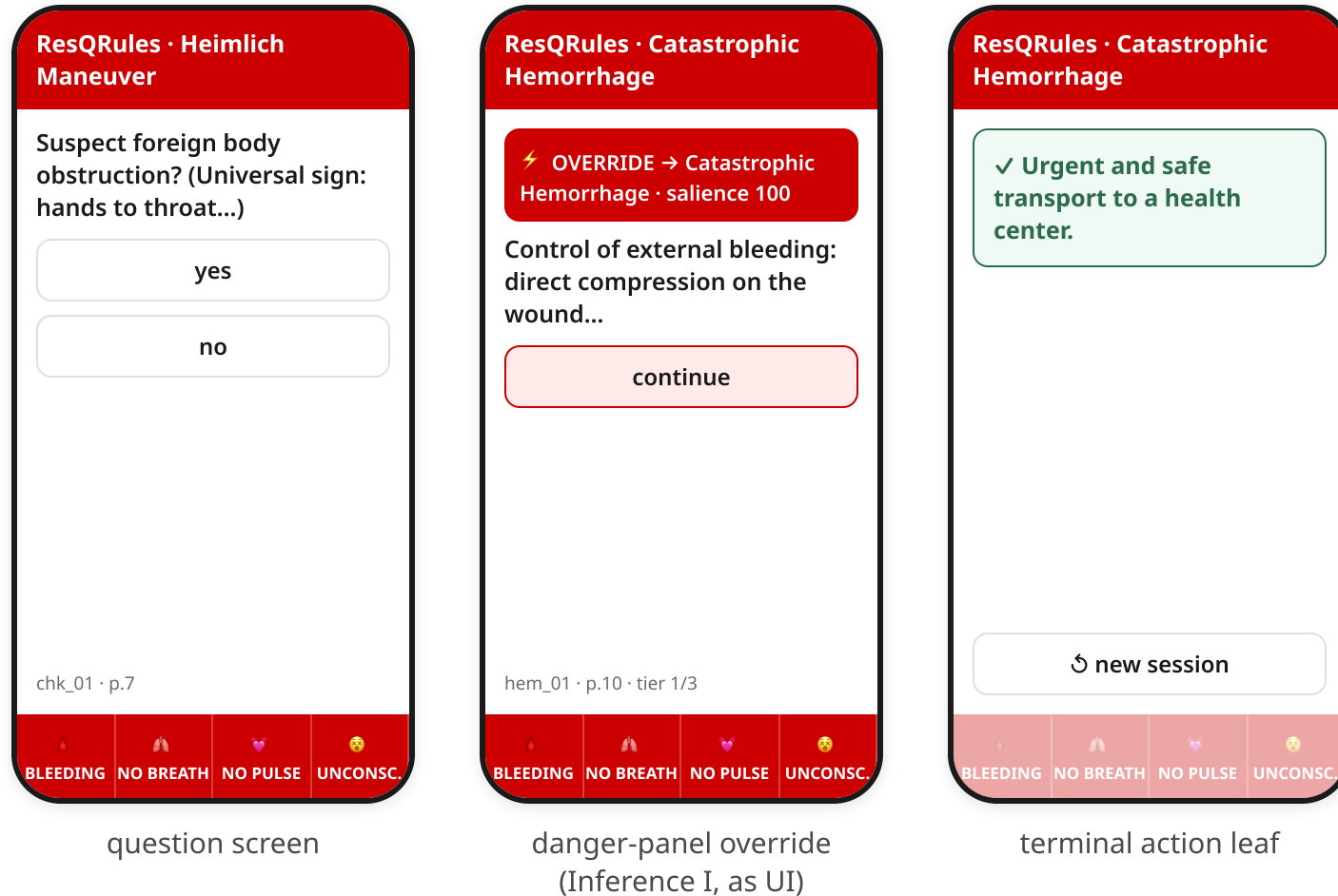
SARC chart #1 · p.1 · data/general_assessment.json
← navigate · F11 full screen · P print/PDF

The API contract is a consequence of the architecture

```
POST /sessions/{id}/step    ← real response (abridged)
{
  "screen": {
    "type": "question",      ← the whole contract
    "node_id": "chk_01",
    "chart_id": "choking",
    "text": { "en": "Suspect foreign body obstruction?...",
              "ar": "هل يُشْتَبِه في انسداد جسم غريب؟" },
    "options": [ {"id":1,"label":{"en":"yes","ar":"نعم"}},
                  {"id":2,"label":{"en":"no","ar":"لا"}} ]
  },
  "danger_panel": [ ...b · n · p · u... ]
}
```

- Response shapes are driven by **node type from the JSON knowledge** — never by internal rules or salience.
- New rules → **zero API change**. New chart → **auto-discovered** by GET /charts.
- Every text field ships **both languages** → switching EN↔AR needs no server round-trip.

The same knowledge, in a responder's pocket



- One widget per server node type — **the KBS drives the UI**; the shell never changes.
- The persistent **danger panel** = the salience overrides, as UI.
- Instant EN↔AR — both languages arrive in every response; RTL flips locally.
- Stack, in one breath: Clean Architecture · Cubit · Retrofit.

Live demo

The engine narrates its own inference — watch the salience fire.

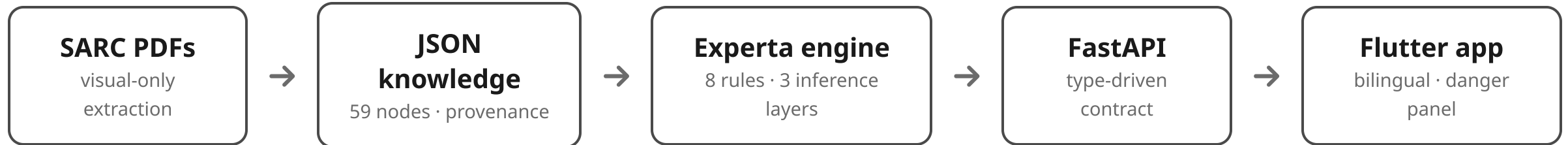
danger key b mid-assessment → sal-100 preempt

"unsure" at Sign of life? → CF routes to compressions

the app, in the field

(presenter: switch to the video / live terminal now)

From paper to a reasoning system



**Paper can't react.
Knowledge as data + a small inference engine can.**

honest limits: jumps are one-way (no return-stack yet) · Phase-2 charts are JSON-only additions

Knowledge → engine → API → app



KNOWLEDGE

M.GH

Software engineer with paramedic experience · knowledge source & reference collector · Flutter dev



ENGINE

M.SH

Python expert · AI engineer — the Experta inference engine



API

M.JA

Backend engineer — wrapped the engine in FastAPI



APP

M.Y.D

Flutter developer & designer



APP · OPS

M.Marwan

Flutter expert · UI designer · DevOps on the server · Linux lover

Thank you, Miss Omama.

For the guidance that kept the knowledge honest and the inference
real.

Questions?

ResQRules · 8 rules · 59 nodes · 5 protocols